

ROS 2 + PX4: Architecture, Integration & Industrial Applications

Agenda

- Why Communication Matters
- Data FLOW
- Hardware Architecture
- High Level Hardware Architecture
- Software Architecture
- What is happening inside ROS 2 SITL
- Practical Exposure

Ardupilot with ROS2

Introduction — Why Communication Matters

★ Think of it like this: the FC speaks TWO languages.

✓ FC ↔ CC still uses MAVLink for ground control & mission communication

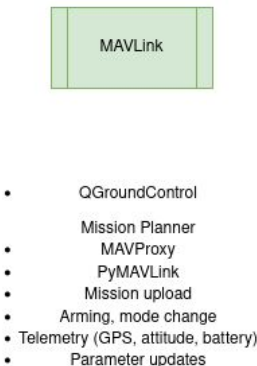
✓ FC also uses XRCE-DDS for ROS2 topic communication

✓ Both run together

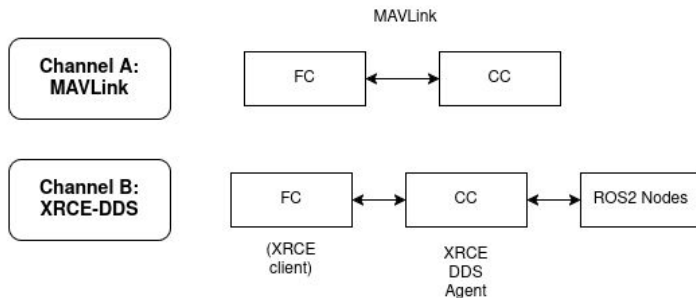
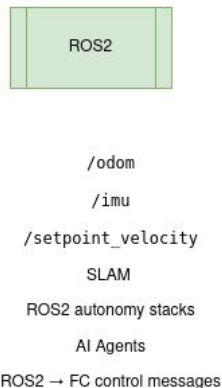
✓ XRCE-DDS does NOT replace MAVLink

✓ MAVROS is not needed anymore for ROS2, but MAVLink is still required for GCS and essentials

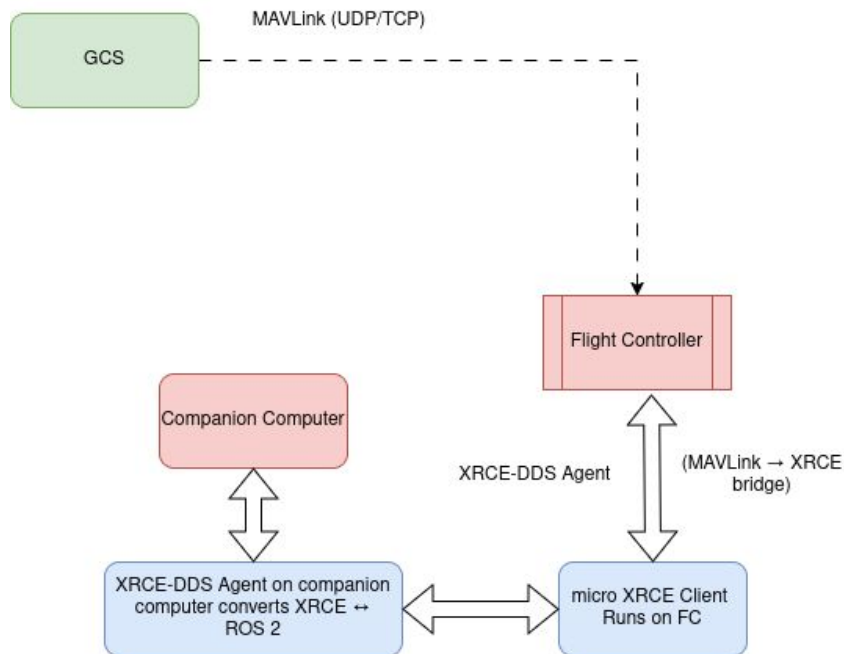
MAVLink - The Original Language



XRCE-DDS — a new language for ROS2



Data Flow between FC - CC- GCS



Sensors → ArduPilot → Control loops.

ArduPilot publishes telemetry using **XRCE-DDS client**.

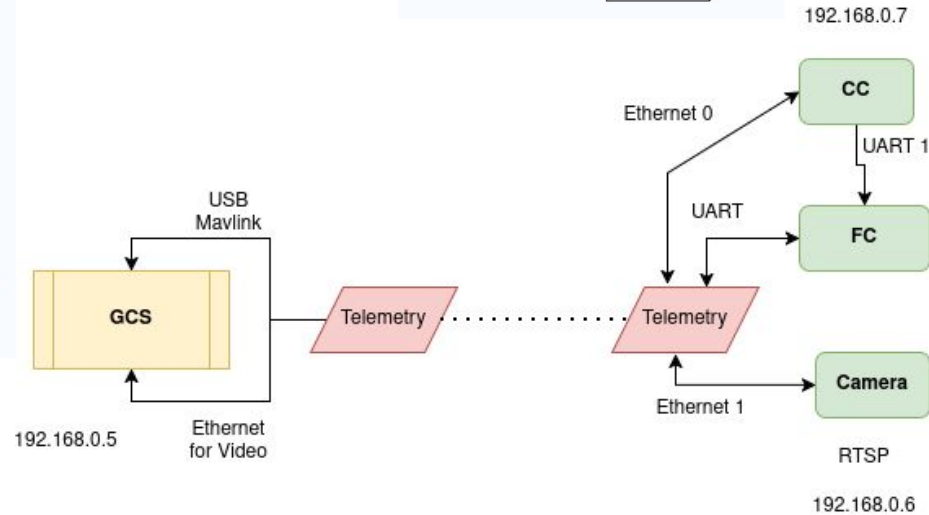
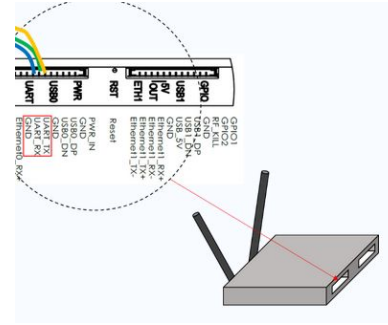
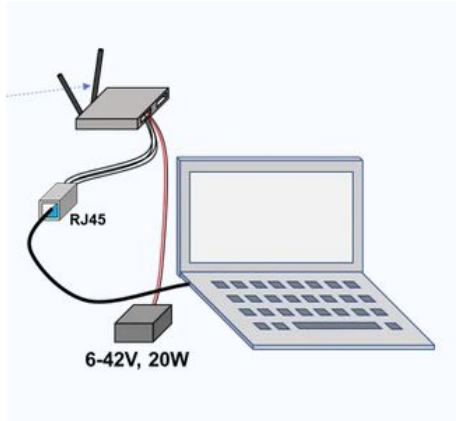
XRCE Agent receives data → exposes them as ROS 2 topics.

ROS 2 nodes process data and create commands.

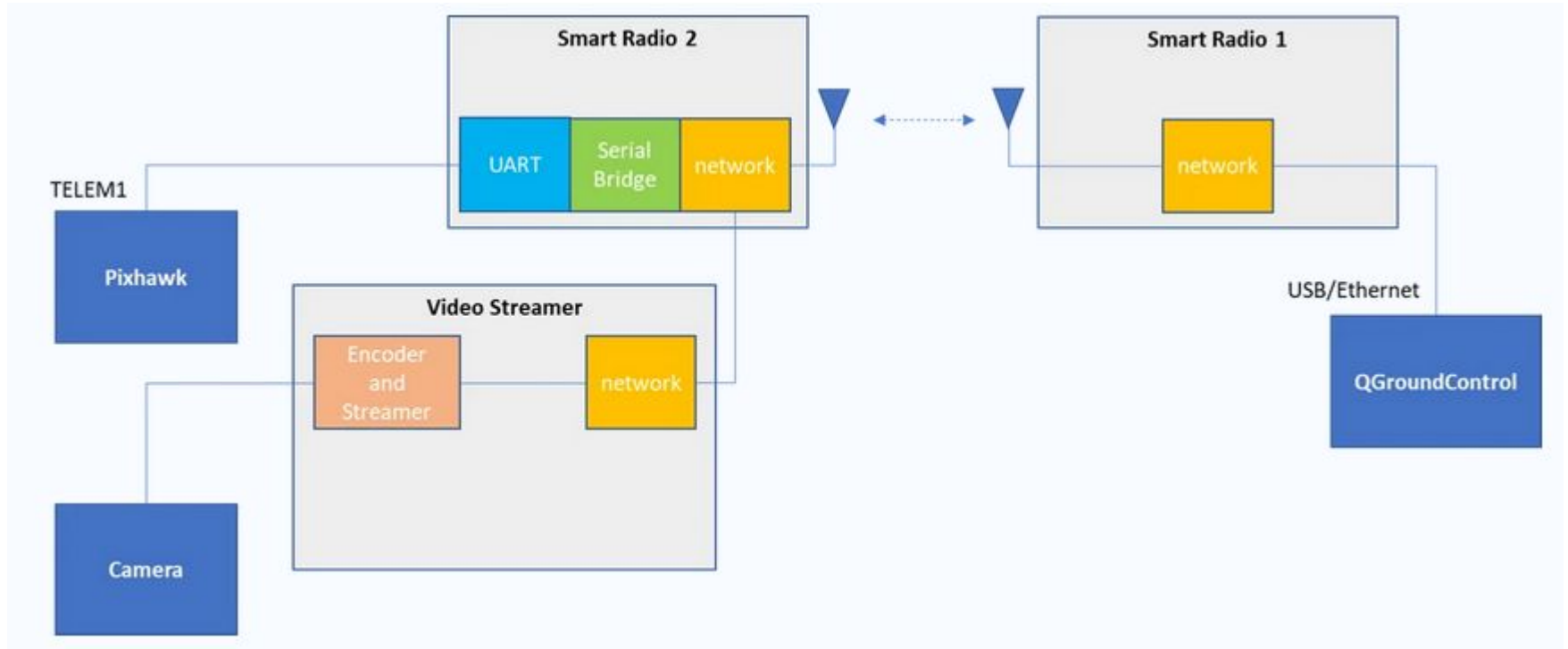
Commands go back through **XRCE Agent → XRCE Client → ArduPilot**.

ArduPilot converts incoming topics to appropriate MAVLink commands.

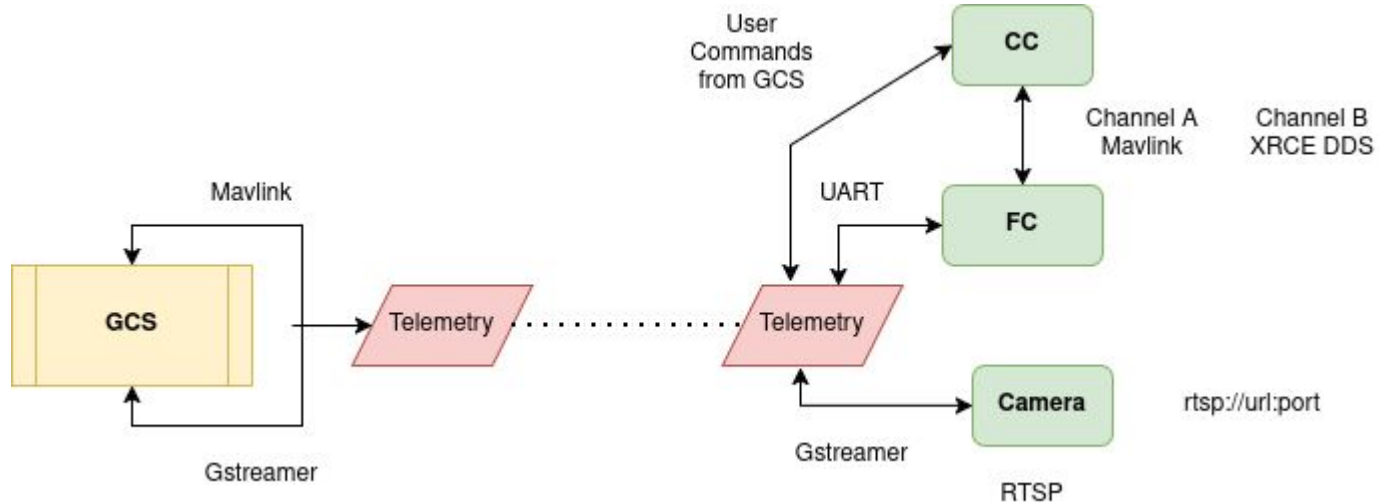
Hardware Communication GCS - CC - FC - Radio



High Level Hardware Architecture FC - GCS

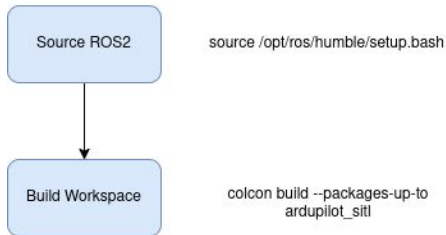


Software Architecture



What is happening inside ROS2 - SITL - Ardupilot

XRCE-DDS — a new language for ROS2



Frame

model:=quad

Starts SITL as:
ArduCopter - Quadrotor

The Main Launch Command

```
ros2 launch ardupilot_sitl sitl_dds_udp.launch.py
```

ArduPilot SITL + DDS bridge + UDP endpoints.

SITL

speedup:=1

Simulates **real time**.

`speedup=10` runs 10× faster.

transport:=udp4

- ArduPilot will use **UDP transport for DDS/XRCE**
- `udp4` = IPv4
- Micro XRCE DDS Client (inside ArduPilot) sends DDS messages over UDP to the Agent.

synthetic_clock:=True

ROS2 will use a **sim time clock**, not wall time.

Needed for SITL simulation accuracy.

Parameters

defaults:=...

Two parameter files loaded:

1. **copter.parm** → base ArduCopter parameters
2. **dds_udp.parm** → enables the XRCE-DDS client

Communication

sim_address:=127.0.0.1

Where SITL outputs its simulation data, e.g., state, sensors.

This is internal-only (localhost).

sitl:=127.0.0.1:5501

This is the internal control socket for SITL to receive messages from the wrapper process.

master:=tcp:127.0.0.1:5760

THIS IS IMPORTANT.

This sets:

ArduPilot SITL MAVLink output → TCP port 5760

Meaning:

SCS

SITL (ArduPilot) → MAVLink → localhost:5760

So QGroundControl or MAVProxy connect here.

This is the GCS link.

Now Lets Launch ROS2 - SITL - Ardupilot

ArduPilot 4.5

ArduPilot 4.6 and later

```
source /opt/ros/humble/setup.bash
cd ~/ardu_ws/
colcon build --packages-up-to ardupilot_sitl
source install/setup.bash
ros2 launch ardupilot_sitl sitl_dds_udp.launch.py \
transport:=udp4 \
synthetic_clock:=True \
wipe:=False \
model:=quad \
speedup:=1 \
slave:=0 \
instance:=0 \
defaults:=$(ros2 pkg prefix ardupilot_sitl)/share/ardupilot_sitl/config/default_params/copy
sim_address:=127.0.0.1 \
master:=tcp:127.0.0.1:5760 \
sitl:=127.0.0.1:5501
```

ROS2 CLI

source ~/ardu_ws/install/setup.bash

ros2 node list

ros2 node info /ap

ros2 topic echo /ap/geopose/filtered

```
/ap
Subscribers:
/ap/cmd_gps_pose: ardupilot_msgs/msg/GlobalPosition
/ap/cmd_vel: geometry_msgs/msg/TwistStamped
/ap/joy: sensor_msgs/msg/Joy
/ap/tf: tf2_msgs/msg/TFMessage
Publishers:
/ap/airspeed: ardupilot_msgs/msg/Airspeed
/ap/battery: sensor_msgs/msg/BatteryState
/ap/clock: roscpp_msgs/msg/Clock
/ap/geopose/filtered: geographic_msgs/msg/GeoPoseStamped
/ap/goal_lla: geographic_msgs/msg/GeoPointStamped
/ap/gps_global_origin/filtered: geographic_msgs/msg/GeoPointStamped
/ap/imu/experimental/data: sensor_msgs/msg/Imu
/ap/navsat: sensor_msgs/msg/NavSatFix
/ap/pose/filtered: geometry_msgs/msg/PoseStamped
/ap/rc: ardupilot_msgs/msg/Rc
/ap/status: ardupilot_msgs/msg/Status
/ap/tf_static: tf2_msgs/msg/TFMessage
/ap/tlme: builtin_interfaces/msg/Time
/ap/twist/filtered: geometry_msgs/msg/TwistStamped
Service Servers:
/ap/arm_motors: ardupilot_msgs/srv/ArmMotors
/ap/experimental/takeoff: ardupilot_msgs/srv/Takeoff
/ap/get_parameters: rcl_interfaces/srv/GetParameters
/ap/mode_switch: ardupilot_msgs/srv/ModeSwitch
/ap/prearm_check: std_srvs/srv/Trigger
/ap/set_parameters: rcl_interfaces/srv/SetParameters
Service Clients:
```

```
header:
  stamp:
    sec: 1763287360
    nanosec: 991462000
    frame_id: base_link
  pose:
    position:
      latitude: -35.36326217651367
      longitude: 149.1652374267578
      altitude: 584.0800170898438
    orientation:
      x: 0.0008026795694604516
      y: 0.00011648357030935585
      z: 0.7446434497833252
      w: 0.6674618721008301
```

ROS2 Topics

```
source ~/ardu_ws/install/setup.bash
```

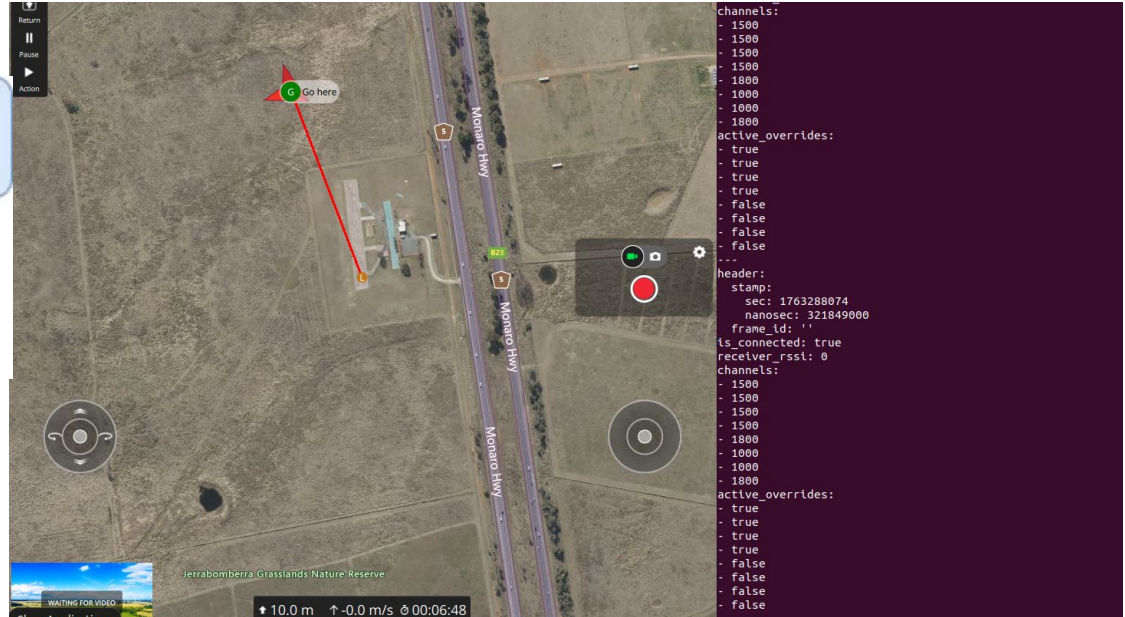
ros2 topic list

```
crond@crond-BELL-G15-5550: ~/ardu_ws$ ros2 topic list  
/ap/airspeed  
/ap/battery  
/ap/clock  
/ap/cmd_gps_pose  
/ap/cmd_vel  
/ap/geopose/filtered  
/ap/goal_lla  
/ap/gps_global_origin/filtered  
/ap/imu/experimental/data  
/ap/joy  
/ap/navsat  
/ap/pose/filtered  
/ap/rc  
/ap/status  
/ap/tf  
/ap/tf_static  
/ap/time  
/ap/twist/filtered  
/parameter_events  
/rosout
```

ROS2 Topics Echo to check RC

```
source ~/ardu_ws/install/setup.bash
```

```
ros2 topic echo /ap/rc
```



ROS 2 Services - ARM



ros2 service call /ap/arm_motors ardupilot_msgs/srv/ArmMotors "{arm: true}"

ROS 2 Services - Change Flight Mode

```
ros2 service call /ap/mode_switch ardupilot_msgs/srv/ModeSwitch "{mode: 4}"
```

Loiter = 5

Altitude Hold = 2

Guided = 4

RTL= 6

Land = 6

ROS 2 Services - Go To Lat Lon

```
ros2 topic pub /ap/cmd_gps_pose ardupilot_msgs/msg/GlobalPosition  
"{header: {frame_id: 'map'}, coordinate_frame: 5, latitude: 40.08370,  
longitude: -105.21740, altitude: 1630.0}"
```

```
ros2 topic pub /ap/cmd_gps_pose ardupilot_msgs/msg/GlobalPosition "{header: {frame_id: 'map'}, coordinate_frame: 5, latitude: 40.08370, longitude: -105.21740, altitude: 1630.0}"
```


ROS 2 Creating Node

Steps

- 1) ~/ardu_ws/src/ardupilot_dds_tests/ardupilot_dds_tests
- 2) Create file cmd_vel_pub.py
- 3) Open - ~/ardu_ws/src/ardupilot_dds_tests/ardupilot_dds_tests/setup.py
- 4) "cmd_vel_publisher = ardupilot_dds_test.cmd_vel_pub:main"

```
entry_points={
    'console_scripts': [
        "time_listener = ardupilot_dds_tests.time_listener:main",
        "plane_waypoint_follower = ardupilot_dds_tests.plane_waypoint_follower:main",
        "pre_arm_check = ardupilot_dds_tests.pre_arm_check_service:main",
        "copter_takeoff = ardupilot_dds_tests.copter_takeoff:main",
        "cmd_vel_publisher = ardupilot_dds_tests.cmd_vel_pub:main",
        "vel_control_publisher = ardupilot_dds_tests.velocityControl:main",
    ],
}
```

Build = colcon build --packages-up-to ardupilot_dds_tests

Run = ros2 run ardupilot_dds_tests cmd_vel_publisher

Commands

- 1) ros2 topic info /ap/cmd_vel
- 2) Ros2 topic echo /ap/cmd_vel

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import TwistStamped

class CmdVelPublisher(Node):
    def __init__(self):
        super().__init__('vel_control_publisher')
        self.publisher_ = self.create_publisher(TwistStamped, '/ap/cmd_vel', 10)
        self.timer = self.create_timer(0.5, self.publish_cmd_vel)

    def publish_cmd_vel(self):
        msg = TwistStamped()
        msg.header.frame_id = 'map'
        msg.twist.linear.x = -1.0
        msg.twist.linear.y = 0.0
        msg.twist.linear.z = 0.5
        msg.twist.angular.x = 0.0
        msg.twist.angular.y = 0.0
        msg.twist.angular.z = 0.0
        self.publisher_.publish(msg)
        self.get_logger().info(f'Publishing TwistStamped: {msg}')

def main(args=None):
    rclpy.init(args=args)
    node = CmdVelPublisher()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```